

# AML Assignment — Project Plan (Revised)

University of Sussex, Spring 2026

## Contents

|                                                                     |          |
|---------------------------------------------------------------------|----------|
| <b>Applied Machine Learning Assignment — Project Plan (Revised)</b> | <b>1</b> |
| 1. How marks are actually won . . . . .                             | 1        |
| 2. Repo and environment . . . . .                                   | 2        |
| 3. Task 1 — Sentiment Analysis (50%) . . . . .                      | 3        |
| 4. Task 2 — Face Alignment (50%) . . . . .                          | 5        |
| 5. Report writing strategy (3000-word limit) . . . . .              | 9        |
| 6. Suggested timeline . . . . .                                     | 10       |
| 7. Submission checklist . . . . .                                   | 10       |
| 8. Tripwires to avoid . . . . .                                     | 10       |
| 9. Originality of thought, within scope . . . . .                   | 11       |

## Applied Machine Learning Assignment — Project Plan (Revised)

**Module:** AML, University of Sussex, Spring 2026 **Constraint:** This revision restricts every method to techniques explicitly taught in the lecture slides on the project. Anything not in the lectures is flagged as out-of-scope.

---

### 1. How marks are actually won

Re-read Section 1 of the brief. The marks are mostly for *understanding, critical thinking and originality*, not raw accuracy. Of the 100 marks:

- **60 marks** — Methods description + Analysis of results (the report)
- **20 marks** — Extension work (NLTK external eval, robustness analysis)
- **20 marks** — Test-set accuracy

So the report does the heavy lifting. A model scoring 78% with sharp, well-justified analysis beats one scoring 84% with a thin writeup.

#### High-grade signals examiners look for:

1. More than one approach compared with the same evaluation harness (the brief says “for top marks, you should consider more than one approach” — twice).

2. Diagrams of pipeline + model architecture (the brief says diagrams are “expected” — twice).
  3. Explicit failure case analysis with plausible explanations.
  4. Identification of systematic bias.
  5. Justified hyperparameter choices, not defaults left untouched.
  6. Properly defined evaluation metrics.
  7. References to literature cited in the lectures (Mikolov et al., Cao et al., Xiong & De la Torre, Newell et al., Jurafsky & Martin, Canny, Lowe).
  8. A declaration of how generative AI was used.
- 

## 2. Repo and environment

One repo, two folders. Reproducibility matters — graders may not run your code, but messy code suggests messy thinking.

```
aml-assignment/
├── README.md
├── requirements.txt
├── data/                                # gitignored
├── task1_nlp/
│   ├── 01_explore.ipynb
│   ├── 02_baseline_wordlist.ipynb
│   ├── 03_spam_handling.ipynb
│   ├── 04_models.ipynb
│   ├── 05_evaluate.ipynb
│   ├── 06_nltk_external.ipynb
│   └── src/
├── task2_cv/
│   ├── 01_explore.ipynb
│   ├── 02_baseline_meanface.ipynb
│   ├── 03_hog_regression.ipynb
│   ├── 04_cnn_direct.ipynb
│   ├── 05_cnn_heatmap.ipynb          # optional — only if 04 leaves headroom
│   ├── 06_robustness.ipynb
│   └── src/
├── report/
├── submission/
│   ├── task1_predictions.csv
│   └── task2_predictions.csv
```

Fix random seeds. Save trained model weights so figures can be regenerated without retraining. Log start/end times — the brief asks for training time.

Compute: a free Colab T4 GPU is fine for both tasks at sensible image resolutions.

---

### 3. Task 1 — Sentiment Analysis (50%)

#### 3.1 The actual problem

Binary sentiment with email-style spam evenly mixed into both training classes. Spam has no true label. At test time, output 0, 1, or a dummy label (e.g. -1) for spam.

Naïvely training a binary classifier on the corrupted training set will learn from noisy labels — a chunk of “negative” examples are spam, and so are a chunk of “positive”. The decision boundary gets pulled toward distinguishing review-vocabulary from spam-vocabulary rather than positive-vs-negative sentiment. That’s the failure mode the assignment is testing, and a great baseline to demonstrate.

#### 3.2 Pre-processing pipeline (W02\_L03)

State and justify each step. The lecture covers all of:

- **Sentence segmentation / word tokenisation** — NLTK regex tokenizer or `split()`.
- **Case normalisation** — lowercase.
- **Number normalisation** — replace digits with a <NUM> placeholder or remove.
- **Punctuation removal** — keep ! and ? if they help sentiment, drop the rest. Discuss the trade-off explicitly.
- **Stopword removal** — discuss the standard NLTK stopwords list and the risk of removing negation words (“not”, “no”) that matter for sentiment. Decide whether to use a reduced list and document why.
- **Stemming** (Porter, NLTK) vs **lemmatisation** (WordNet, NLTK) — try both, compare, justify your final choice. Lecture flags errors of commission/omission for stemming.
- **Bag-of-words representation** — frequency or binary, sparse storage.

A flowchart of this pipeline is required for the methods section.

#### 3.3 Features / representations (W02\_L04, W05\_L09)

Plan at least two so you can compare:

- **Bag-of-words** (binary or frequency).
- **TF-IDF** (W05\_L09) — note the variants (boolean tf, length-adjusted, log-scaled). Pick one and justify.
- **Word embeddings** (W05\_L09) — Word2vec via gensim (CBOW or skip-gram, lecture covers both). Average the word vectors in a document to form a document vector — this is a simple, defensible choice.

You can layer cosine similarity on top of TF-IDF or embeddings (W05\_L09) to do spam handling — see next subsection.

#### 3.4 Spam handling — using only lecture-taught techniques

The lectures don’t cover anomaly detection (Isolation Forest, One-Class SVM, etc.), so build spam handling from what *is* covered:

**Approach A — Cosine similarity to class centroids (W05\_L09).** 1. Compute TF-IDF (or averaged-embedding) vectors for every training document. 2. Compute the centroid

(mean vector) of the documents labelled positive and the centroid of documents labelled negative. These centroids are noisy because of the spam, but the *bulk* of each class is real movie reviews, so the centroid still points roughly the right way. 3. For each document, compute cosine similarity to both centroids. A document with low similarity to *both* centroids is unlike either type of movie review — likely spam. 4. Pick a threshold (sweep it on validation, plot a curve) below which documents are tagged as spam.

This is entirely within W05\_L09 and gives you a clean, principled spam filter to justify.

**Approach B — Word-list spam filter (W02\_L04).** The lecture covers word-list classifiers: positive list  $L_+$ , negative list  $L_-$ , classify by counting matches. Extend this: 1. Build  $L_+$  (most frequent words in declared-positive documents) and  $L_-$  (most frequent words in declared-negative documents) using either the most-frequent-terms method or the greatest-frequency-difference method (both in W02\_L04). 2. Build a movie-vocabulary list  $L_M = L_+ \cup L_-$ . 3. Documents with very few  $L_M$  matches are unlikely to be movie reviews  $\rightarrow$  tag as spam.

Same idea as Approach A but in word-count space rather than vector space — useful as a comparison.

**Approach C — Noise-tolerant classifier with no explicit spam stage.** Naive Bayes (W03\_L05) is reasonably robust to label noise because per-feature class conditionals are estimated independently. Train Naive Bayes on the full corrupt training set with three “predicted” outputs by post-hoc thresholding the predicted probability: if  $\max(P(+|d), P(-|d))$  is below some threshold  $\tau$ , output the dummy label. Discuss this as the simplest possible approach.

### 3.5 Sentiment models (W02\_L04, W03\_L05, W04\_L08)

Compare at least two:

- **Word-list classifier** (W02\_L04) — required as the simplest baseline. Score with positive minus negative word counts vs a margin  $\delta$ . The lecture gives the exact formula.
- **Naive Bayes** (W03\_L05) — generative, trained by MLE on  $P(c)$  and  $P(f|c)$ . Multinomial NB is the natural choice for text. State the model assumption (feature independence given class) and why it’s a “naive” assumption that still works.
- **Logistic regression** (W03\_L05) — discriminative, trained with cross-entropy loss (the negative log-likelihood; the lecture frames this via the sigmoid). State the loss function explicitly.
- **RNN with embeddings** (W04\_L08) — many-to-one architecture (lecture diagram), input is the sequence of word embeddings, hidden state at the final timestep feeds a classifier head. Discuss vanishing gradients and the LSTM mitigation (W04\_L08 covers both). Only do this if you have time and want to demonstrate a deep-learning approach; the lecture explicitly contrasts RNN pros and cons.

For each model, state in the report: - ML task (binary / 3-way classification with dummy). - Model family. - Loss function (likelihood for NB; cross-entropy for LR; cross-entropy at final step for RNN). - Hyperparameters and how chosen (validation-set sweeps).

### 3.6 Evaluation (15 marks) — W02\_L04, W03\_L05

Required: - **Confusion matrix** — required by brief. Make it 3×3 once you have a spam decision. Lecture (W02\_L04) gives the 2×2 form; extend it. - **Accuracy and error rate** (W02\_L04 formulas). - **Precision, recall, F1** per class (W02\_L04 formulas; F1 is the harmonic mean of P and R). - **Precision-recall trade-off** discussion (W03\_L05 covers this) — show a curve where you sweep the spam threshold from Approach A.

Qualitative: - Pick 5–10 misclassified validation examples per error type, read them, write 1–2 sentences each on *why* the model failed. - Identify systematic patterns: short reviews? Negation-heavy reviews (“not bad”)? Reviews mentioning negative themes positively (“a brilliant portrayal of grief”)? These are the failure cases the brief asks for.

Bias to look for: - Vocabulary genre effects (horror reviews use sentiment-loaded words descriptively). - Length bias (short documents are harder for both classifiers and the spam filter). - Spam false positives on legitimate but unusual reviews. - Class imbalance effects (W02\_L04 raises this explicitly).

### 3.7 NLTK external evaluation (10 marks)

```
from nltk.corpus import movie_reviews
```

Run your chosen final model. The corpus has no spam, so: - Your spam detector should fire ~0% of the time. If it fires more, that’s a generalisation failure of the spam stage — discuss. - Sentiment accuracy will likely drop vs your validation set (older reviews, different distribution). Discuss why: vocabulary drift, domain shift, label-noise differences.

### 3.8 Test set submission (10 marks)

- Predict {0, 1, dummy} where dummy is documented in the report (e.g. -1).
- Do not reorder the test data.
- Use the provided “save as csv” function in the Colab worksheet.
- Sanity check: open the CSV, count rows, check label distribution.

### 3.9 Suggested figures for Task 1 report

- Pre-processing flowchart.
  - Spam-handling flowchart.
  - TF-IDF + cosine-similarity diagram (centroids and threshold).
  - Confusion matrix per approach.
  - Precision-recall trade-off curve for the spam threshold.
  - 3–4 captioned failure-case examples.
- 

## 4. Task 2 — Face Alignment (50%)

### 4.1 The actual problem

5 landmarks (eyes, nose, mouth corners; Figure 2 in brief). Direct coordinate regression. The “twist” is variability — likely rotations, scaling, lighting, possibly noise. Build a robust system.

You cannot use dlib or mediapipe face alignment. Face *detection* is not required.

## 4.2 Pre-processing (W06\_L12, aml\_computervision\_toolkit, W08\_L15)

- **Resize / interpolation** — to  $64 \times 64$  or  $128 \times 128$ . The brief explicitly says you don't need full resolution. Critically, transform the landmark coordinates by the same factor.
- **Grayscale conversion** — reduces input channels from 3 to 1, often fine for landmark regression.
- **De-meaning and normalisation** (W06\_L12) — pixel intensities to  $[0, 1]$  floats; optionally subtract per-image or per-dataset mean.
- **Histogram equalisation** (W06\_L12) — improves global contrast. Lecture flags it can amplify noise. Try with and without and report which works.

A flowchart of this pipeline is expected.

## 4.3 Features / representations (W09\_L17, W09\_L18, W10\_L19)

Plan at least two:

- **Raw / flattened pixels** — for the simplest baseline.
- **HOG descriptors** (W09\_L17) — the lecture covers HOG in detail:  $6 \times 6$  cells, orientation histograms over  $0-180^\circ$  (invariant to edge polarity),  $3 \times 3$  block normalisation for local-contrast invariance. Note explicitly that HOG is approximately invariant to brightness changes and small rotations — that's exactly the robustness story you'll want for the variability twist.
- **CNN features** (W10\_L19) — for the deep-learning approach, the network learns its own features. Lecture covers convolutional layers, channels, receptive fields, invariance vs equivariance.

(SIFT is also covered in W09\_L17 if you prefer it for patch-based features around landmark estimates — the lecture explicitly compares SIFT vs HOG.)

## 4.4 Approaches to compare (plan three)

**Baseline 0 — Mean face (W09\_L18 implicit).** Predict the mean training landmark positions for every test image. The lecture explicitly points out that face shape is low-dimensional and can be approximated with PCA — the mean is the 0-basis approximation, which is your floor.

**Approach 1 — Linear regression from HOG features.** 1. Compute HOG over the resized image (W09\_L17 method). 2. Linear regression (multi-output) from HOG vector to 10 outputs ( $5 \text{ landmarks} \times 2$ ). MSE loss (the squared-error / Euclidean-distance objective in W09\_L18). 3. Optionally use PCA on the shape vectors (W09\_L18) to regress in shape-parameter space and reconstruct landmarks — the lecture shows shape-parameter regression as one of the two regression options.

**Approach 2 — Cascaded regression with pose-indexed features (W09\_L18).** Lecture covers this directly: 1. Initialise from the mean shape. 2. At each stage, compute feature descriptors (HOG patches) at the *current* landmark estimates — these are pose-indexed features (Cao et al., explicitly cited in W09\_L18). 3. Regress the residual (offset

to the true shape) from these features. 4. Iterate K stages. The lecture frames this as supervised descent (Xiong & De la Torre).

The lecture also mentions ensembles of regression trees (Kazemi & Sullivan, the link in W09\_L18) as one implementation — sklearn's GradientBoostingRegressor or RandomForestRegressor would let you implement this.

This is the most “lecture-faithful” classical approach and a strong choice if you prefer not to lean on deep learning.

**Approach 3 — CNN direct coordinate regression (W09\_L18, W10\_L19).** 1. Small CNN: a few conv blocks (kernel 3×3, the lecture's worked example), pooling, flatten or global average pool, fully-connected head with 10 outputs. 2. Loss: MSE / squared Euclidean — the W09\_L18 objective  $f(x) = ||h(x) - y||^2$ . The lecture flags that “more robust loss functions might be required” for direct prediction; you can mention L1 / Huber as alternatives but the lectures don't go further than this, so MSE is the defensible primary choice. 3. Train with SGD / Adam (Adam is the standard practical default; mention it as such).

**Approach 4 (optional) — CNN heatmap regression (W09\_L18).** Lecture covers this as the second main deep-learning approach: predict one heatmap per landmark, target is a Gaussian blob at the true location, take soft-argmax (lecture explicitly mentions “the arg-max is not differentiable, so need to create heatmap images as targets or use soft-argmax”). The hourglass network (W11\_L21, Newell et al.) is the lecture-cited architecture; a single hourglass block is enough for 5 landmarks.

Pick three. A solid combination is **Mean face + HOG-regression cascade + CNN direct regression**. If you want to swing for the top marks, add the heatmap variant.

#### 4.5 Loss function — what the lectures sanction

W09\_L18 names the loss as  $f(x) = ||h(x) - y||^2$  — that's L2 / MSE / squared Euclidean. Stick to this for direct regression, with one optional sentence that the lecture acknowledges “more robust loss functions might be required” without specifying which — this is your rationale for trying a single alternative if you want, but don't go beyond what's safe to cite.

For heatmap regression, the natural target is a 2D Gaussian and the natural loss is pixel-wise MSE on the predicted vs target heatmap.

#### 4.6 Data augmentation (W09\_L18 — explicitly for face alignment, and W10\_L20)

This is your main tool against the variability twist. W09\_L18 lists exactly: - Random rotations - Scales - Translations - Brightness / contrast changes - Left / right flips — **with annotation flips** (the lecture explicitly warns: “make sure you flip the annotations!”). For 5 landmarks: eyes swap (indices 0 ↔ 1), mouth corners swap (3 ↔ 4), nose stays (2).

W10\_L20 adds blur, colour balance, vignette, pincushion distortion as further augmentations.

Apply on the fly during training with a deterministic transform that takes image + landmarks together and returns the transformed pair. A flowchart of the augmentation pipeline is required by the brief if you augment.

#### 4.7 Evaluation (15 marks) — W09\_L18

Required by brief: - **Cumulative Error Distribution (CED)** curve. x: normalised error threshold; y: % of validation images below that threshold. One line per approach on the same axes. - **Boxplots** per approach.

Metric (W09\_L18 explicit): - **Euclidean distance** between predicted and ground-truth landmark. - **Normalised by inter-eye distance** (between landmarks 0 and 1) — this removes the effect of image resolution and face size in the image, exactly as the lecture says.

Define this in the report as a numbered equation.

Per-landmark error breakdown — the lecture explicitly says “the simplest statistic we could report is the mean error in our predicted points. This tells a very limited story” and asks what else you could report. Give per-landmark mean and median, and per-landmark CED.

Qualitative: - Show validation images with predicted (one colour) and ground-truth (another colour) landmarks overlaid. 3 successes, 3 failures. - Failure analysis: extreme rotation? Heavy distortion? Unusual pose? Lighting?

#### 4.8 Robustness analysis (10 marks)

Pick one approach and stress-test on the validation set:

- **Gaussian noise**: add at increasing  $\sigma$ , plot CED at each level. The brief suggests this directly.
- **Rotations**: apply increasing rotation, plot CED.
- **Scaling**: similar.
- **Contrast / brightness changes**: similar.

Then compare a model **trained with augmentation** vs **trained without**. The augmented model should degrade more gracefully — that’s the story.

Discuss *why* robustness exists where it does, using lecture concepts: - Convolution gives translation equivariance (W10\_L19 covers this directly). - HOG’s invariance to brightness changes and small rotations (W09\_L17) explains why HOG-based approaches are more robust to those specific perturbations than raw-pixel approaches. - Augmentation teaches invariance to seen perturbations — W10\_L20 explicitly: “used data augmentation to help teach the network transformations”. - Heatmap regression tends to be more robust than direct coordinate regression because errors are spatially local — mention this if you implemented it.

#### 4.9 Test set submission (10 marks)

- Output shape: (n\_test\_images, n\_points, 2).
- Coordinates in **original image resolution**, not training resolution. Track your scaling factor.
- Use the provided “save as csv” function.
- Don’t reorder.



#### 4.10 Suggested figures for Task 2 report

- Pre-processing flowchart.
  - Augmentation flowchart.
  - Architecture diagram for each model (especially the CNN).
  - CED plot, all approaches on one axes.
  - Boxplot per approach.
  - Per-landmark error bars.
  - Robustness CED curves at increasing perturbation levels.
  - 3 success + 3 failure example images.
- 

#### 5. Report writing strategy (3000-word limit)

Two sections, one per task. Suggested allocation:

| Section                     | Words                    |
|-----------------------------|--------------------------|
| Task 1 — Methods            | 600                      |
| Task 1 — Results & analysis | 500                      |
| Task 1 — NLTK external eval | 250                      |
| Task 2 — Methods            | 600                      |
| Task 2 — Results & analysis | 500                      |
| Task 2 — Robustness         | 250                      |
| References                  | (excluded)               |
| <b>Total</b>                | <b>~2700</b> with buffer |

Per-task writing checklist:

- ☐ Pipeline diagram (pre-processing + features)
- ☐ Model architecture diagram
- ☐ Justification paragraph for *every* major design choice, citing the relevant lecture
- ☐ Hyperparameter table with values and how chosen
- ☐ Compute usage line (hardware + training time)
- ☐ Confusion matrix (T1) / CED + boxplot (T2)
- ☐ Quantitative comparison table across approaches
- ☐ At least 3 named failure cases with explanations
- ☐ Bias discussion paragraph
- ☐ Generative AI declaration

References to cite (all from the lectures): Mikolov et al. 2013 (Word2vec); Cao et al. (Explicit Shape Regression); Xiong & De la Torre (Supervised Descent Method); Newell et al. 2016 (Stacked Hourglass); Canny 1986 (edge detection); Lowe (SIFT); Kazemi & Sullivan (ensemble of regression trees) if used; Jurafsky & Martin 2026 (cited throughout for embeddings and similarity).

---

## 6. Suggested timeline

**Week 1 — Get end-to-end** - Days 1-2: Repo set-up, data loaded for both tasks, sanity-check shapes. - Days 3-4: Task 1 baselines (word-list classifier, plus a naïve Naive Bayes ignoring spam). Submission CSV writes correctly. - Days 5-7: Task 2 baseline (mean face) and a simple HOG + linear regression. Submission CSV writes correctly.

**Week 2 — Improve** - Days 1-3: Task 1 spam handling (cosine-to-centroid + word-list filter). Train NB and LR on cleaned data. - Days 4-6: Task 2 cascaded regression with pose-indexed HOG features, and/or a small CNN. - Day 7: Hyperparameter tuning, save best model weights.

**Week 3 — Extend, evaluate, write** - Day 1: Task 1 NLTK external eval. - Day 2: Task 2 robustness analysis curves. - Day 3: Regenerate all figures at publication quality (font sizes, axis labels, captions). - Days 4-6: Write report. Two passes: first dump everything, then cut to 3000 words. - Day 7: Final packaging — zip, file naming, double-check CSV row counts and order.

Buffer day at the end.

---

## 7. Submission checklist

- ☐ Report PDF,  $\leq 3000$  words (excluding references)
  - ☐ Notebooks / .py files included or linked via public GitHub repo
  - ☐ task1\_predictions.csv — produced by provided save-as-csv function, correct shape, correct order, dummy label documented
  - ☐ task2\_predictions.csv — provided save-as-csv function, shape (n\_images, n\_points, 2), original-image-resolution coordinates, correct order
  - ☐ Original datasets **not included**
  - ☐ .zip only
  - ☐ Generative AI usage declaration in report
- 

## 8. Tripwires to avoid

- **Reordering test data.** The brief warns about this twice.
  - **Predicting at the wrong resolution for Task 2.** Scale predictions back to original image size before saving.
  - **Forgetting to flip landmark indices on horizontal flip.** W09\_L18 calls this out specifically.
  - **Removing “not” with a default stop-word list** — destroys sentiment signal.
  - **Using only lecture-taught techniques** — this revised plan is built around that. If you find yourself reaching for a method, ask: “which lecture and slide?” If you can’t answer, swap it for one you can.
  - **Reporting only your best model.** The brief explicitly wants comparisons.
  - **Skipping the Generative AI declaration.** Required by the brief.
-

## 9. Originality of thought, within scope

The brief lists this as a graded skill. Things that are *original* but still strictly within taught material:

- For Task 1: framing spam handling as a cosine-similarity-threshold problem in TF-IDF space, with the threshold chosen by sweeping a precision-recall trade-off curve (W03\_L05) on the validation set.
- For Task 1: comparing Naive Bayes' robustness to label noise vs Logistic Regression's, and showing both confusion matrices to make the point quantitatively.
- For Task 1: comparing TF-IDF features vs averaged Word2vec embeddings as inputs to the same classifier, with the lecture's argument (W05\_L09) about why dense embeddings can outperform sparse vectors.
- For Task 2: a clean ablation of HOG vs raw pixels at the same regression model, showing HOG's invariance to brightness/rotation paying off in the robustness curves (W09\_L17 plus the robustness analysis).
- For Task 2: comparing cascaded regression (with pose-indexed features per W09\_L18) against a CNN trained from scratch, with a discussion of inductive biases (lecture-cited explicitly: convolution gives equivariance, cascading constrains the search trajectory).
- For Task 2: showing that augmentation flattens the robustness degradation curve, demonstrating exactly the W10\_L20 claim that augmentation "teaches" the network transformations.

You don't need all of these. One or two per task, executed carefully and tied back to specific lecture content, is the sweet spot.

---

*Plan revised 2026-05-10. Every method named here can be traced to a specific lecture slide on the project. Verify against your own notes before writing the report — the references you cite need to come from your reading, not this plan.*